Distributed Gaming Backends

Carl Borngrund

September 3, 2026



Contents for today

State synchronisation models

Lag compensation techniques

- Client-side prediction
- Server reconciliation
- Latency hiding

Architectures

- Peer-to-peer vs central server architectures




Terran Barracks
MENU






$2500
100

Video Games

Long history

- Started late 50s, early 60s
- Exploded in the 70s

Single physical device

- Arcade, NES, N64, PlayStation

Non-networked games

- Ocarina of Time, Super Mario, FFVII

Early online multiplayer games

- Counter-Strike, Halo, DOOM









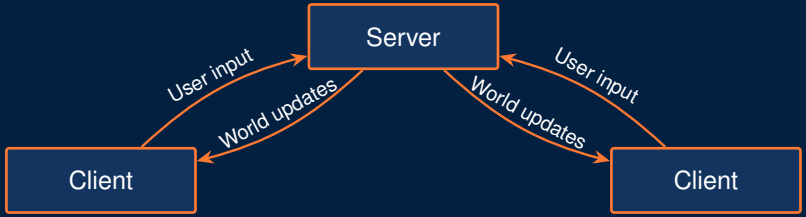


What makes an (online) game multiplayer?

Group of running instances share a common state

- The state is updated by input

State must be kept synchronised among participants



Why is online gaming difficult?

(Soft) real-time systems

- System must deliver a reaction in fixed intervals

Relatively tight time constraints

- Depends on the chosen rate (tick)
- Most games are above 30 Hz at least

Potentially a very large number of participants

Unreliable networks

Heterogeneous hardware

Consistency

Security

Networked Game Genres

Examples

- First person shooters (FPS)
- Massively multiplayer online role-playing games (MMORPG)
- Real-time strategy (RTS)
- Turn-based strategy games
- Fighting games
- Multiplayer online battle arena (MOBA)

All have different requirements, leading to different challenges

- Trade-offs!

System Requirements

Game genre	Latency tolerance	State size	Consistency model	Tick rates
FPS	Very low	Small (position, ammo)	Hybrid, server-authoritative	60–128 Hz (CS2, Valorant)
RTS	Low	Large (many units, map state)	Lockstep, eventual consistency	10–30 Hz
MMORPG	Medium	Very large (inventory, quests, world state)	Eventual consistency, server-side logic	10–20 Hz
Fighting	Extremely low	Tiny (player inputs only)	Deterministic lockstep, rollback	Variable (with frame delay)
Turn-based	Very high	Medium	Full consistency	No real tick rate
MOBA	Low–Medium	Medium (skills, cooldowns, creeps)	Server-authoritative + reconciliation	30–60 Hz

System Requirements

Game genre	Latency tolerance	State size	Consistency model	Tick rates
FPS	Very low	Small (position, ammo)	Hybrid, server-authoritative	60–128 Hz (CS2, Valorant)
RTS	Low	Large (many units, map state)	Lockstep, eventual consistency	10–30 Hz
MMORPG	Medium	Very large (inventory, quests, world state)	Eventual consistency, server-side logic	10–20 Hz
Fighting	Extremely low	Tiny (player inputs only)	Deterministic lockstep, rollback	Variable (with frame delay)
Turn-based	Very high	Medium	Full consistency	No real tick rate
MOBA	Low–Medium	Medium (skills, cooldowns, creeps)	Server-authoritative + reconciliation	30–60 Hz

Latency Tolerance

What is latency?

- Time delay between cause and effect
- One-way trip time
- Round-trip delay time
- For gaming: often only measuring the network, not the visual update

Latency varies between pairs of players

- Depending on location and connection

Game genre	Latency tolerance
FPS	Very low
RTS	Low
MMORPG	Medium
Fighting	Extremely low
Turn-based	Very high
MOBA	Low–Medium

Latency Tolerance

Example: SF to NY

- Speed of light (300000 km/s): latency is 13 ms
- In fibre, slower: $\sim$210000 km/s ($\rightarrow$ 19 ms)
- Routers, switches etc. — say 30 ms
- State update in game — say 100 ms
- Plus getting state into the backend and out to clients again
- Total possible round trip $\sim$200 ms

We are bound by the speed of light through fibre

Game genre	Latency tolerance
FPS	Very low
RTS	Low
MMORPG	Medium
Fighting	Extremely low
Turn-based	Very high
MOBA	Low–Medium

Latency Tolerance

What is latency tolerance?

- The tolerance we have to latency, or lag, to ensure quality of service of the game
- Tolerance is different between the different game genres

We use many tricks to hide latency from the players — we will get into that in a bit

Game genre	Latency tolerance
FPS	Very low
RTS	Low
MMORPG	Medium
Fighting	Extremely low
Turn-based	Very high
MOBA	Low–Medium

System Requirements

Game genre	Latency tolerance	State size	Consistency model	Tick rates
FPS	Very low	Small (position, ammo)	Hybrid, server-authoritative	60–128 Hz (CS2, Valorant)
RTS	Low	Large (many units, map state)	Lockstep, eventual consistency	10–30 Hz
MMORPG	Medium	Very large (inventory, quests, world state)	Eventual consistency, server-side logic	10–20 Hz
Fighting	Extremely low	Tiny (player inputs only)	Deterministic lockstep, rollback	Variable (with frame delay)
Turn-based	Very high	Medium	Full consistency	No real tick rate
MOBA	Low–Medium	Medium (skills, cooldowns, creeps)	Server-authoritative + reconciliation	30–60 Hz

State Size

What is state size?

- The amount of data that has to be tracked
  $\rightarrow$ Synchronised

Why is state size important?

- Bandwidth usage
- Update rate
- Scalability
- Performance

Game genre	State size
FPS	Small (position, ammo)
RTS	Large (many units, map state)
MMORPG	Very large (inventory, quests, world state)
Fighting	Tiny (player inputs only)
Turn-based	Medium
MOBA	Medium (skills, cooldowns, creeps)

State Size

Why does state size differ between game genres?

- Different gameplay needs
- World is tracked differently
- World is simulated differently
- Updates are communicated differently

We have to understand the needs of the game in order to minimise the state size, and so minimise data over the wire!

Game genre	State size
FPS	Small (position, ammo)
RTS	Large (many units, map state)
MMORPG	Very large (inventory, quests, world state)
Fighting	Tiny (player inputs only)
Turn-based	Medium
MOBA	Medium (skills, cooldowns, creeps)

System Requirements

Game genre	Latency tolerance	State size	Consistency model	Tick rates
FPS	Very low	Small (position, ammo)	Hybrid, server-authoritative	60–128 Hz (CS2, Valorant)
RTS	Low	Large (many units, map state)	Lockstep, eventual consistency	10–30 Hz
MMORPG	Medium	Very large (inventory, quests, world state)	Eventual consistency, server-side logic	10–20 Hz
Fighting	Extremely low	Tiny (player inputs only)	Deterministic lockstep, rollback	Variable (with frame delay)
Turn-based	Very high	Medium	Full consistency	No real tick rate
MOBA	Low–Medium	Medium (skills, cooldowns, creeps)	Server-authoritative + reconciliation	30–60 Hz

Consistency Models

A consistency model defines the rules that determine how to deal with changes in the game state by distributed applications

Examples

- Eventual consistency
- Rollback netcode

Game genre	Consistency model
FPS	Hybrid, server-authoritative
RTS	Lockstep, eventual consistency
MMORPG	Eventual consistency, server-side logic
Fighting	Deterministic lockstep, rollback
Turn-based	Full consistency
MOBA	Server-authoritative + reconciliation

Consistency Models

Different game genres have different requirements and priorities.

Choosing the incorrect consistency model can make it very difficult to reach those requirements.

Game genre	Consistency model
FPS	Hybrid, server-authoritative
RTS	Lockstep, eventual consistency
MMORPG	Eventual consistency, server-side logic
Fighting	Deterministic lockstep, rollback
Turn-based	Full consistency
MOBA	Server-authoritative + reconciliation

Consistency Models

Examples:

- Fighting games require consistency within a single number of frames
- Without this, it will be hard to reach competitive fairness

Game genre determines:

- How fast updates must happen
- How precise the game state must be
- How scalable the system must be

Game genre	Consistency model
FPS	Hybrid, server-authoritative
RTS	Lockstep, eventual consistency
MMORPG	Eventual consistency, server-side logic
Fighting	Deterministic lockstep, rollback
Turn-based	Full consistency
MOBA	Server-authoritative + reconciliation

System Requirements

Game genre	Latency tolerance	State size	Consistency model	Tick rates
FPS	Very low	Small (position, ammo)	Hybrid, server-authoritative	60–128 Hz (CS2, Valorant)
RTS	Low	Large (many units, map state)	Lockstep, eventual consistency	10–30 Hz
MMORPG	Medium	Very large (inventory, quests, world state)	Eventual consistency, server-side logic	10–20 Hz
Fighting	Extremely low	Tiny (player inputs only)	Deterministic lockstep, rollback	Variable (with frame delay)
Turn-based	Very high	Medium	Full consistency	No real tick rate
MOBA	Low–Medium	Medium (skills, cooldowns, creeps)	Server-authoritative + reconciliation	30–60 Hz

Tick Rate

What is tick rate?

- A tick is one full cycle of the game server's game loop
- Tick rate is how many times the game server updates the game state per second

For example:

- Tick rate 60 Hz (60 updates a second)
- One update every $\sim$16.67 ms

Game genre	Tick rates
FPS	60–128 Hz (CS2, Valorant)
RTS	10–30 Hz
MMORPG	10–20 Hz
Fighting	Variable (with frame delay)
Turn-based	No real tick rate
MOBA	30–60 Hz

Tick Rate

Why is tick rate important?

- Direct impact on the game's performance with regard to the requirements

Higher tick rate:

- Lower input latency, better hit detection, smoother movement

Trade-offs:

- Higher server load, higher bandwidth usage, increased cost

Game genre	Tick rates
FPS	60–128 Hz (CS2, Valorant)
RTS	10–30 Hz
MMORPG	10–20 Hz
Fighting	Variable (with frame delay)
Turn-based	No real tick rate
MOBA	30–60 Hz

Tick Rate

Why is it different between game genres?

- Different requirements
  - How fast is the game
  - How many players are simulated
  - What kind of consistency is needed
  - Etc. . .

Trade-off: gameplay quality vs cost

Game genre	Tick rates
FPS	60–128 Hz (CS2, Valorant)
RTS	10–30 Hz
MMORPG	10–20 Hz
Fighting	Variable (with frame delay)
Turn-based	No real tick rate
MOBA	30–60 Hz

Architectural Models

Peer-to-peer

Client-server

Server networks

Architectural Models: Peer-to-peer

Beginning of the 90s (Doom)

Less used now, but is an option (e.g. in Unity)

Decentralised: every client sends to everybody

Everything synchronised (inputs, outputs)

Every client executes the same code on the same input and gets the same results (determinism)

Architectural Models: Peer-to-peer

Trade-offs:

+ Easy to introduce (no extensive server backend)
+ Every player has the same environment

- Required bandwidth grows polynomially
- Hard to join/leave at will (state responsibilities)
- Network problems can be fatal to many
- Every client is trusted (very easy cheating)

Architectural Models: Client-server

One central server controls the game

Clients deliver user input and receive updates from/to the server only

Semi-synchronous updates

Clients may have little or no game logic

Architectural Models: Client-server

Trade-offs:

+ Better network scalability
+ Lower bandwidth requirements (linear increase)
+ Cheating (a little) harder

- Network latency is disruptive
- Server load
- Clients/server trust each other (security issues)

Architectural Models: Hybrid/Authoritative

"Client-server architectures enhanced"

- Clients use prediction algorithms
- Clients maintain a subset of the game logic

Trade-offs:

+ Reduced network bandwidth
+ Latency or transmit errors can be handled
+ Improved response

- Consistency

But what about really large games? (MMOs)

Playfield must be partitioned

Multiple servers required

Clients receive only ‘relevant’ information

- Updates about local surroundings
- Player moves in a bubble of attention

Different strategies exist for how to filter information

Architectural Models: Server Networks

Two approaches:

- Distribute clients
  - Clients connected to one server
  - Servers exchange information about clients
- Partition game world
  - Every server is responsible for a section of the world
  - Clients move from server to server when moving

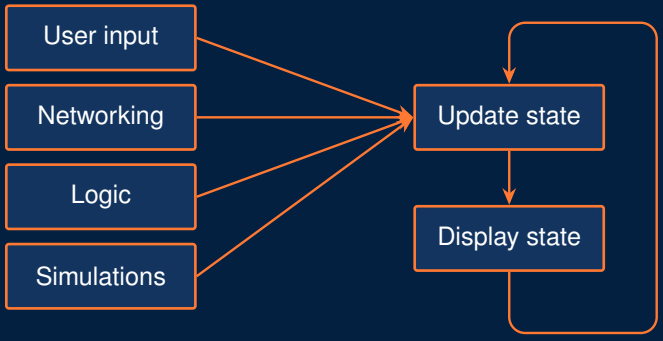
Architectural Models: Server Networks

Two approaches — and what they cost:

- Distribute clients
  - Higher network load among servers
  - Latency
- Partition game world
  - Uneven distribution of workload (all clients in one section. . .)

What do all games have in common?

We have a game loop



State Synchronisation

What is state synchronisation?

- The process of ensuring all players in a multiplayer game see a consistent game world
- The world needs to be consistent independent of device, latency, and geographical location

Without state synchronisation

- Players would see different realities
- Some action could be valid on player A's screen but not on the server
- The gameplay would suffer

State Synchronisation

Why can’t we just send the full state and keep it consistent?

- Bandwidth limitations
  - Modest-sized games will quickly become kilobytes or megabytes
- Latency and congestion
  - Packet delay or loss with larger states
- Redundancy
  - Most information does not change every tick
- Scalability
  - When there are too many entities, servers cannot afford to send a full state to thousands of players every tick
- Security
  - Sending the full state gives every client information about the full world

State Synchronisation

State synchronisation techniques depend on the game

Different game genres, different requirements

Some examples:

- Snapshot interpolation
- Delta compression
- Entity interpolation
- Deterministic lockstep

State Synchronisation: Deterministic Lockstep

Send inputs, simulate everything else on the clients.

How it works:

- Each player joins the game and has the exact same start state
- The game is fully simulated on each client
- For frame *n*:
  - Players send all their inputs to all others on frame *n*
  - All clients wait until they receive information about frame *n*
  - Once they receive the information, each client simulates frame *n*
  - Repeat

Used in RTS games such as StarCraft

State Synchronisation: Deterministic Lockstep

Advantages:

+ Low bandwidth requirements
+ Full consistency

Limitations:

- Requires determinism
- Have to wait for every input from every player
- Players cannot join late

What if we want more players, or a more dynamic joining situation?

State Synchronisation: Snapshot Interpolation

Clients receive snapshots of the game state periodically from a server and render an interpolated view.

How it works:

- Server sends timestamped snapshots of the game state at some interval
- Clients buffer those snapshots
- Every rendered frame, the client:
  - Finds two snapshots: the most recent before the current rendered time, and the one after
  - Interpolates between them to show a smooth change
- Client is slightly behind real time, but smoothly updates between states

State Synchronisation: Snapshot Interpolation

Advantages:

+ Looks good even with low tick rates
+ No need to wait for input for every frame
+ Very low input lag

Disadvantages:

- Much higher bandwidth requirements
- Introduces a delay
- Prediction more complex

Used in shooters and MOBAs

Talking About Interpolation

What do we do when we detect errors between our clients?

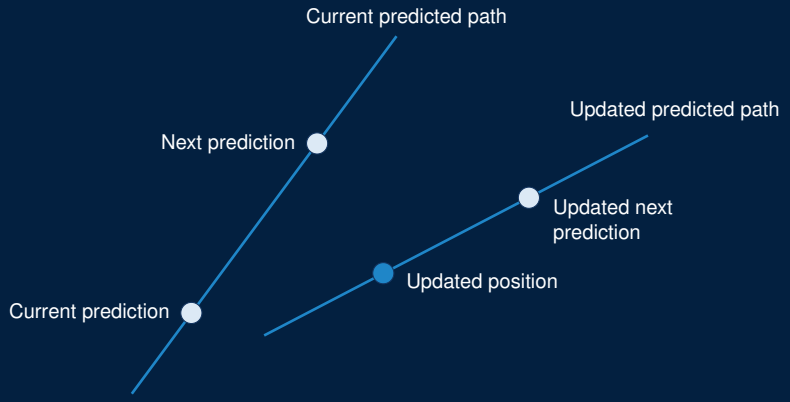
Well, we need some convergence strategy

We want to:

- Correct the errors fast
- Avoid visual artefacts
- Do the calculations within a very short deadline (before the next frame)
  - Use polynomials

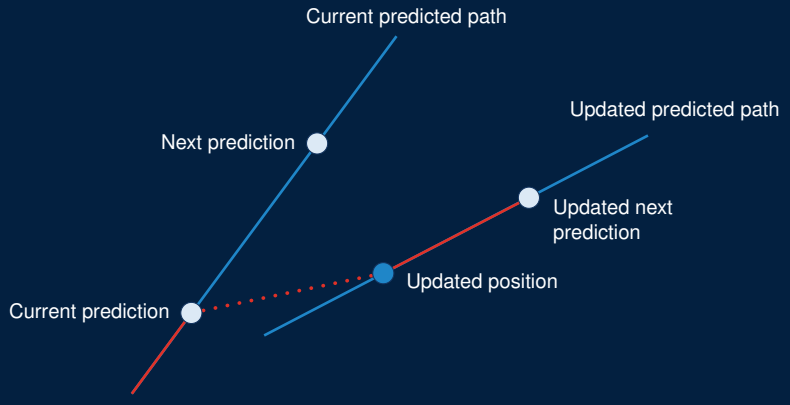
Convergence Strategies

Zero order (snap)



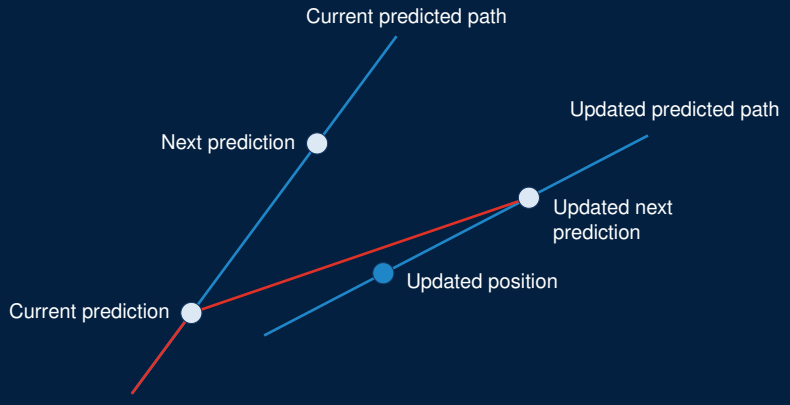
Convergence Strategies

Zero order (snap)



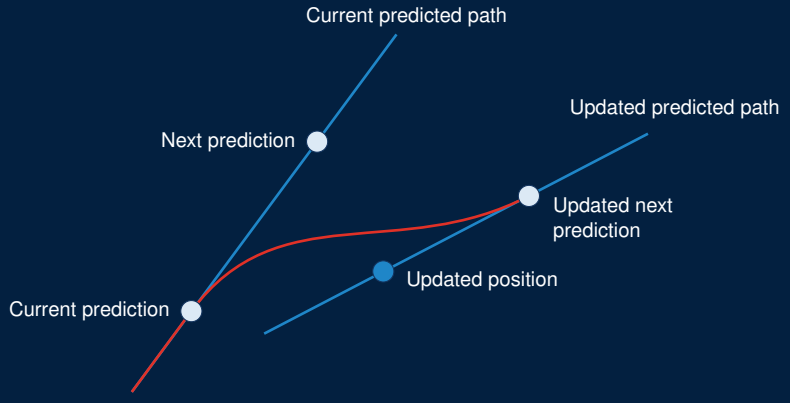
Convergence Strategies

First order (linear)



Convergence Strategies

Higher order



What is lag compensation?

In online gaming, we know that lag will always be a problem

We can't enforce that everyone playing together is in the same physical location

What can we do? Lag compensation

Lag compensation is about making player actions fair despite latency

This can be achieved through:

- Client-side prediction
- Server reconciliation
- Lag compensation techniques
- Latency hiding
- etc.

Lag Example

We have a client-server architecture

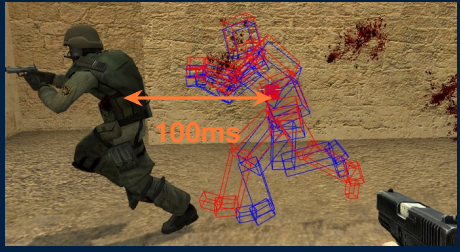
Player A, playing at 50 ms, shoots player B, who is playing at 150 ms

Player B is moving while player A is shooting

Best case: there is a 100 ms difference between what player A sees and where player B actually is

By the time the shot reaches the server, it would no longer hit player B

Player A would always miss the shot (on the server), even if it seemed impossible to miss on their screen



Lag Compensation: Server-Side Rewind

We fix this by doing some type of lag compensation

The server “rewinds” to the world state at the moment the user acted

Steps:

1. Estimate the player’s latency
2. Find the last world update the player saw before issuing the command
3. Rewind other players to where they were at that moment (adjust for latency + interpolation)
4. Execute the user’s command (e.g. shooting, movement) in that past state
5. Restore all entities to their current positions

In this technique, the acting player has the “advantage”

Latency Hiding

Goal: mask network delay to maintain a smooth user experience

- Essential in real-time multiplayer games with frequent state updates

Technique:

- **Dead reckoning** — predicts object/player motion between updates to reduce visible lag

Trade-off:

- Hides latency, but may introduce temporary inconsistencies with server state

Latency Hiding: Dead Reckoning

Predicts future position based on last known state and movement vector.

Formulas:

- Velocity (v):

$$v(t) = s_0 + v\,t$$

- Acceleration (a):

$$s(t) = s_0 + v_0\,t + \tfrac{1}{2}a\,t^2$$

Updates only sent when the actual position deviates beyond a threshold

Balances smooth gameplay with reduced bandwidth usage

Widely used in FPS, racing and simulations

Scaling Online Games

The scale of games has increased massively over the past 20 years

Consumer assumptions about how big a player base a successful game should have have increased a lot

How do we deal with this?

How do we scale these types of games?

What techniques can be used that have minimal impact on gameplay quality?

Scaling Online Games

Depending on the game, a game server can perhaps handle a few hundred players only

But there might be thousands of players

For these large games, we can partition the “stage” (game field) so that each game server only calculates what happens in some partition/area

- E.g. rooms, quadrants of the map, regions, etc.

The world server(s) keep track of where players are and allocate areas to each game server

Partitioning is critical to scaling



What if partitioning isn't enough?

Example scenario:

- New MMO with zones, all users ending up in a few or the same partition
  - Recent release: Classic World of Warcraft

How can we scale this?

- We can add more hardware, but that stops scaling quite quickly, as processors are not really getting that much faster

Other idea: sharding

"Sharding"

We put virtual versions of the partition on top of each other

We divide the players between those

- Allowing us to further partition the player base

Requirement:

- We can't have a game where all players **must** be able to see each other
- When needed, players should be able to move between the partitions

Blizzard calls this technology "sharding"

- Not to be mixed up with database sharding

"Sharding"

Behaviour of sharding:

When too many players are in an area, the game automatically creates a new “shard” — a copy of that area. Players entering the area will be placed on the new shard. The world appears identical from the player’s perspective, with minor differences:

- Players can only see other players on the same shard
- Each shard has its own set of NPCs
- Each shard has its own set of objects, such as resources

Shards can be any geographic size, but are usually limited to zones or smaller areas. The borders of a sharded area are invisible to players.

https://wowpedia.fandom.com/wiki/Sharding_(term)

"Sharding"

Benefit:

- 5x the server population
- Dynamically spin up more partitions when needed

Trade-offs:

- Worse gameplay experience

Probably worth it for a game where a large draw has to do with the population size

This will, however, stop scaling too, at some point

- Some things require complete consistency
- Transactions per second

Communication Between Partitions

Clients keep a connection to the world server

- Which knows the position of the player, directs the player to the right game server, and hands over to another game server as the player moves across the partitions of the stage

Clients keep a connection to a game server

- As directed by the world server
- All the rest of the actual game logic goes on in the game server (a.k.a. area server)

Clients are knowledgeable parts of the distributed system

But how to move between areas?

The game is set up so that you can only move to adjacent areas, and according to some rules (e.g. doors, limited speed, etc.)

The state of all objects that could move to an adjacent area is broadcast, so that the adjacent area server knows where they are already before they step over

- E.g. a pub/sub

Protocols

Relevant protocols are UDP and TCP

Both make sense, depending on the application

UDP vs TCP

TCP

- Connection-oriented
- All packets arrive, even in order
- Fancy features (congestion control, retransmit, . . .)
- Nagle's algorithm
  - Collects data until a reasonable packet size is reached
  - Creates delay

UDP vs TCP

UDP

- Connection-less
- Packets arrive — if at all — in random order
- No feedback about the connection
- Client has to do all the high-level work

UDP vs TCP

TCP is attractive and easy. But!

- Larger overhead
- Latency spikes
- Less control

UDP vs TCP

In games, usually only the current state is of interest

For games, UDP is the weapon of choice

- Used pretty much everywhere

Packet Compression

Computers are fast enough to compress in real time

Reduces network bandwidth

Adds latency

Network errors potentially more fatal

Programming choices

Packet Compression

Commonly used methods:

- Compress individual packets with conventional lossy or lossless methods
  - Huffman, arithmetic coding, quantisation, wavelets
- Use incremental updates
  - Only differences to previous packets are sent (delta)
  - Requires a reliable network, or compensation

Message Aggregation

Merge subsequent packages

- Save bandwidth (header: 28/40 bytes)
- May avoid congestion

Adds latency

- Packets are queued until some size/timeout is reached

Depending on the application, a feasible option

Security & Cheating

Data is sent/received over a network

Typical network precautions apply

Be careful with strings, buffers

Never blindly trust any data

Security & Cheating

Never underestimate the resourcefulness of a cheater

Security through obscurity will eventually always fail

If money is involved (gambling, credit cards, etc.) the fun ends

Security & Cheating

Cheaters can tamper with:

- The network
  - Results in modified packets
- The client
  - Creates legal packets with illegal content
- The client computer
  - Modify drivers
  - Bots, scripts

Security & Cheating

Never trust any data

- Sanity check all received values
- In extreme cases, monitor game progress and check for irregularities

Encrypt packages, add checksums (MD5)

- Avoid tampering with the packages

Sequence numbers

- Prevent a replay attack

Add random data in encrypted packets

Ring-zero anti-cheats

Conclusion

Multiplayer online games are difficult to implement

Many requirements that are in direct opposition to the geographical spread of players

Requirements differ between genres

Requirements decide design choices

Incorrect design choices will make it impossible to provide a good QoS

All design choices have trade-offs!

More Reading

- https://gafferongames.com/#posts
- https://mas-bandwidth.com/
- https://developer.valvesoftware.com/wiki/Latency_Compensating_Methods_in_Client/Server_In-game_Protocol_Design_and_Optimization


LULEÅ
UNIVERSITY
OF TECHNOLOGY